

— MODULARIZACIÓN

SUBPROGRAMAS

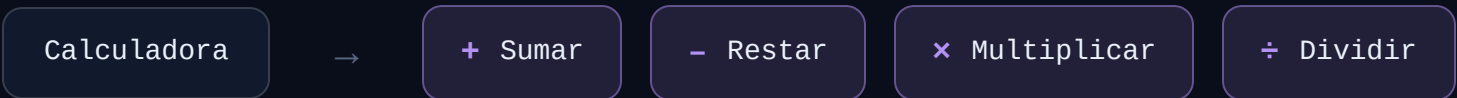
función — devuelve un valor · **procedimiento** — solo actúa

Son bloques de código independientes y reutilizables que realizan una tarea específica dentro de un programa principal. Son la base para escribir código limpio, reutilizable y fácil de mantener.

- dividen el problema en tareas pequeñas
- se pueden reutilizar las veces que haga falta
- 2 tipos: funciones y procedimientos

>_ por qué dividir el problema

Ejemplo: una calculadora



En vez de escribir un programa gigante y enredado, se divide en **subprogramas pequeños** — uno por cada operación. Cada uno se entiende, se prueba y se reutiliza por separado.

>_ los 2 tipos

Función vs. procedimiento

FUNCIÓN devuelve un valor – Realiza una tarea específica – Puede recibir parámetros – Sí devuelve un valor con "Devolver" – Ejemplo: sumar dos números y devolver el resultado	PROCEDIMIENTO no devuelve nada – Realiza una acción específica – Puede recibir parámetros – No devuelve ningún valor – Solo cumple su tarea — por ejemplo, mostrar un mensaje
--	---

>_ cómo se escribe

Sintaxis de una función

```
<tipo de dato> función <nombre>(parámetro)
Inicio
.....
.....
Devolver (valor a retornar)
Fin_función
```

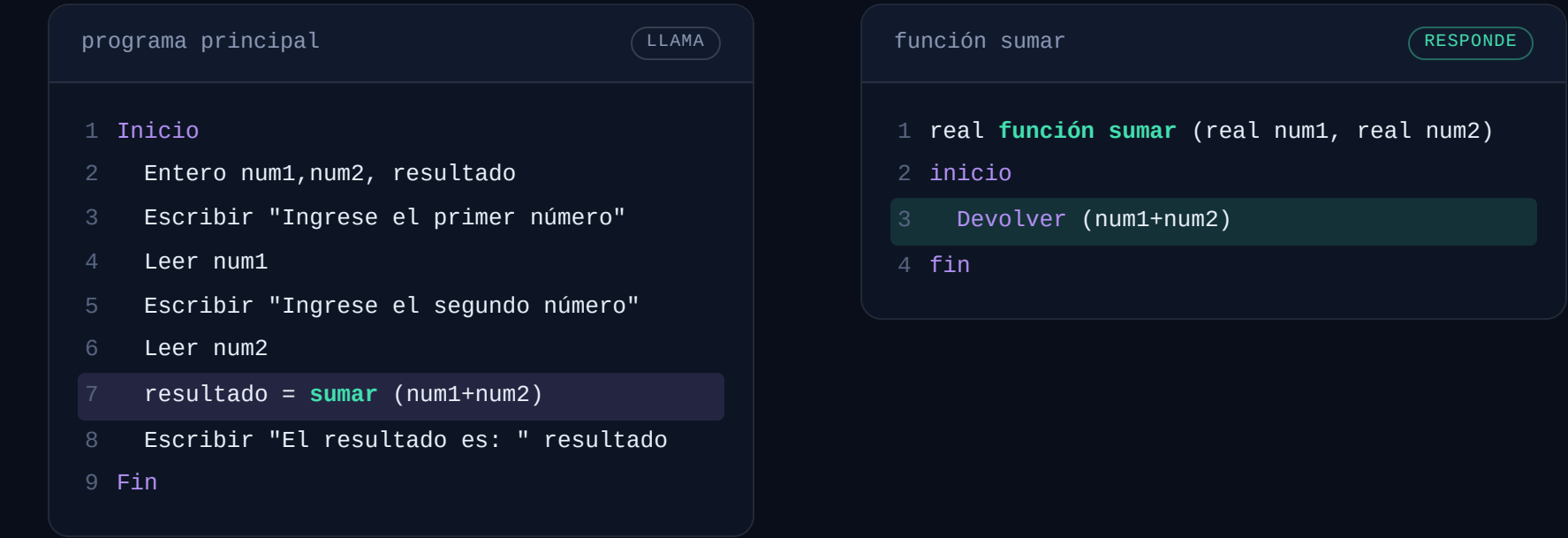
tipo de dato Es el tipo del valor que la función va a **devolver** — por ejemplo, entero o real.

nombre (parámetro) El nombre identifica la función, y el **parámetro** es el dato que recibe para trabajar.

Devolver Entrega el resultado a quien llamó la función. Es lo que distingue a una función de un procedimiento.

>_ ejemplo completo

La función sumar, en acción



📦

Bloques reutilizables
Un subprograma se escribe una vez y se puede llamar desde muchas partes del programa principal.

↩

Función = devuelve algo
Termina siempre con "Devolver", entregando un valor a quien la llamó.

✓

Procedimiento = solo actúa
Cumple su tarea (mostrar, guardar, calcular internamente) pero no entrega ningún valor de vuelta.